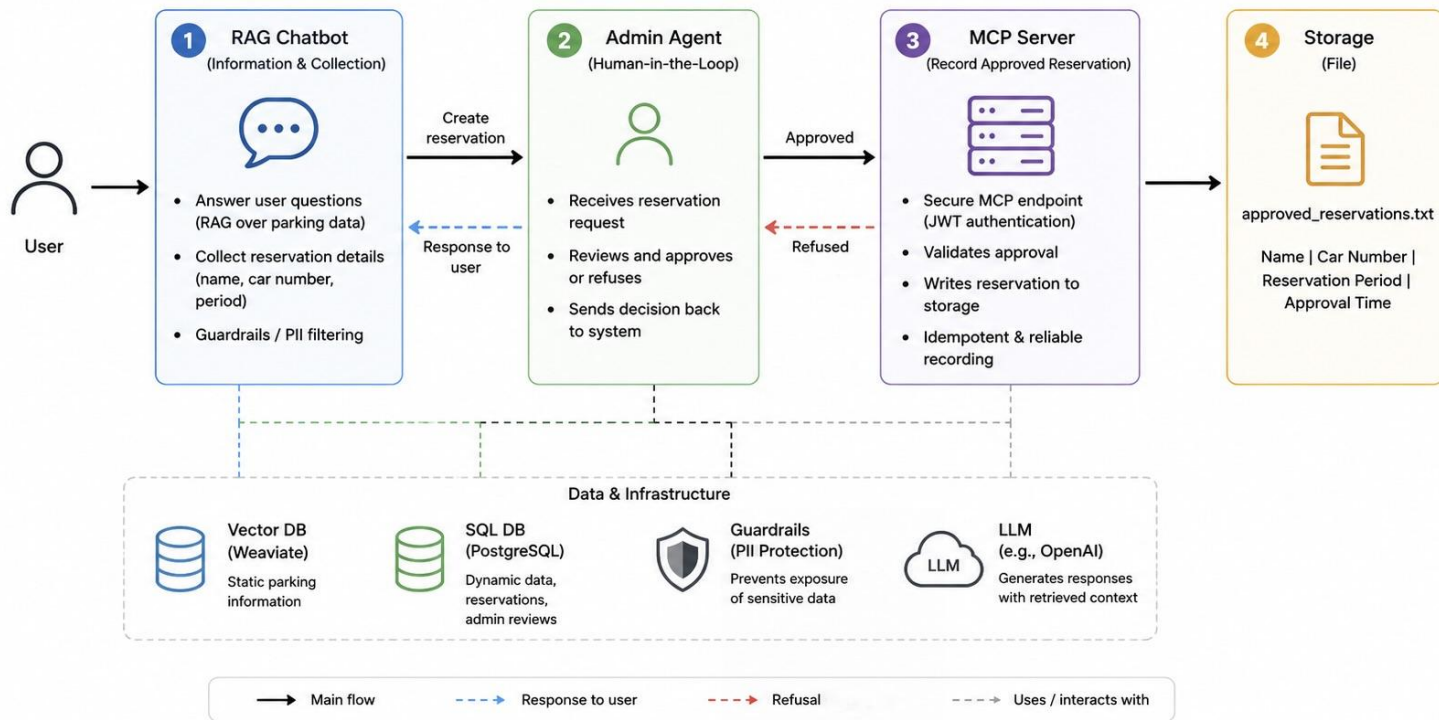


Parking Reservation Chatbot

AI Engineering Fast Track. Practical task
by Viktoriia Tsymbal

Architecture



Preparing the data

Seed PostgreSQL and Weaviate:

Static data → Weaviate

7 knowledge-base documents:

- general parking info
- location
- parking zones
- EV and accessible parking
- booking requirements
- cancellation policy
- privacy policy

Dynamic data → PostgreSQL

- parking spaces and availability
- working hours
- hourly prices
- reservations created by users
- admin reviews (added later in stage 2)

The ingestion command:

- creates PostgreSQL tables if they do not exist
- seeds parking spaces, availability, prices and working hours
- redacts detected PII from static documents
- recreates the Weaviate collection when --reset is used
- indexes 7 static documents in the ParkingInfo collection

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task копія % python -m src.ingest --reset
```

```
Ingested 7 static documents into Weaviate class 'ParkingInfo'
```

Stage 1. RAG chatbot

Static-information questions:

Location

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
CityPark assistant ready. Type 'exit' to quit.
you: Where are you located?
bot: CityPark is located at 12 Market Street, downtown, two minutes' walk from the central train station and accessible from the main ring road.
you: █
```

General parking info

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: Tell me about CityPark
bot: CityPark is located at 12 Market Street, downtown, just a two-minute walk from the central train station and accessible from the main ring road. It offers secure covered and open-air parking spaces in the city center, with 24/7 CCTV surveillance and on-site security staff. CityPark has four zones: A (standard), B (covered), C (long-term), and VIP (closest to the entrance, with valet option), each with different hourly pricing. The available pricing is $2.5/hour for Zone A, $3.0/hour for Zone B, and $6.0/hour for Zone VIP. The working hours are Monday to Friday from 07:00 to 23:00, Saturday from 08:00 to 22:00, and Sunday from 08:00 to 20:00.
you: █
```

Available parking zones

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: What parking zones do you have?
bot: CityPark has four zones: A (standard), B (covered), C (long-term), and VIP (closest to the entrance, with valet option).
you: █
```

Stage 1. RAG chatbot

Static-information questions:

**Electric
charging**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: Do you have electric vehicle charging?
bot: Yes, electric vehicle charging stations are available in Zone B.
you: █
```

**Disabled
parking**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: Do you have disabled parking spaces?
bot: Yes, disabled parking spaces are available in Zone A near the main entrance.
you: █
```

**Required
reservation info**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: What information do I need to make a reservation?
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: █
```

Stage 1. RAG chatbot

Dynamic SQL questions:

**Working
hours**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
CityPark assistant ready. Type 'exit' to quit.
you: What are your working hours on Monday?
bot: The working hours on Monday are 07:00-23:00.
you: █
```

**Available
parking
spaces**

```
you: What parking spaces are currently available?
bot: Available spaces are in Zone A ($2.5/hour), Zone B ($3.0/hour), and Zone VIP ($6.0/hour).
you: █
```

Stage 1. RAG chatbot

Successful **multi-line** reservation:

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: I want to reserve a parking space.
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: Anna
bot: To complete your reservation I still need: your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: Ponomarenko
bot: To complete your reservation I still need: your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: AA5678AA
bot: To complete your reservation I still need: the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-07-13 12:00
bot: To complete your reservation I still need: the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-07-13 14:00
bot: Thanks Anna, your reservation request for Zone A was created. The reservation is pending administrator approval. Reservation reference: 5cd59cc0-8aba-4e9c-905f-8593b21947ca
you: █
```

Successful **one-line** reservation:

```
bot: The current reservation process has been cancelled. You can start a new reservation whenever you are ready.
you: I want to make a reservation
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: My name is Anna Ponomarenko, AA5678AA, 2026-08-25 10:30, 2026-08-25 13:30
bot: Thanks Anna, your reservation request for Zone A was created. The reservation is pending administrator approval. Reservation reference: b257cec7-2ae2-4b3c-84b4-12ffcb13b61b
you: █
```

- all necessary fields collected
- reservation reference generated
- pending administrator approval

Stage 1. RAG chatbot

Edge cases:

**Reset
conversation**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
bot: Thanks Anna, your reservation request for Zone A was created. The reservation is pending administrator approval. Reservation reference: 5cd59cc0-8aba-4e9c-905f-8593b21947ca
you: cancel
bot: The current reservation process has been cancelled. You can start a new reservation whenever you are ready.
you: █
```

**Invalid
reservation
field
(here name as
an example)**

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task konia % python -m src.main
you: I want to reserve a parking space.
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: V
bot: To complete your reservation I still need: your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: Tsybal
bot: To complete your reservation I still need: your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: AA5678AA
bot: To complete your reservation I still need: the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-07-13 12:00
bot: To complete your reservation I still need: the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-07-13 14:00
bot: The first name must contain at least 2 letters and may only contain letters, apostrophes or hyphens.
you: Viktoriia
bot: Thanks Viktoriia, your reservation request for Zone B was created. The reservation is pending administrator approval. Reservation reference: b9ad4299-7b7f-44c9-90dd-29d1cf2963f9
you: █
```

Stage 1. RAG chatbot

Edge cases:

Past date

```
bot: To complete your reservation I still need: the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-06-01 12:00
bot: To complete your reservation I still need: the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-06-01 14:00
bot: The reservation start time must be in the future.
you: █
```

End earlier than start

```
you: I want to reserve a parking space
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: My name is Anna Ponomarenko, car number AA5678AA, from 2026-07-13 14:00 to 2026-07-13 12:00
bot: The reservation end time must be later than the start time.
you: █
```

Start and end are equal

```
you: I want to book a parking space
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: My name is Anna Ponomarenko, car number AA5678AA, from 2026-07-13 12:00 to 2026-07-13 12:00.
bot: The reservation end time must be later than the start time.
you: █
```

Stage 1. RAG chatbot

Edge cases:

**Reservation
crosses
calendar days**

```
you: I want to book a parking space
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: My name is Anna Ponomarenko, car number AA5678AA, from 2026-07-13 08:00 to 2026-07-14 10:00.
bot: A reservation must start and end on the same calendar day.
you: █
```

**During
non-working
hours**

```
you: I want to book a space
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: My name is Anna Ponomarenko, car number AA5678AA, from 2026-07-13 06:00 to 2026-07-13 08:00.
bot: The requested period is outside the parking facility's working hours (07:00-23:00).
you: █
```

Stage 1. RAG chatbot

Reservation-status lookup:

Existing ID

```
you: check 102daa4b-3798-42c0-ab78-22bba40eff9a
bot: Reservation 102daa4b-3798-42c0-ab78-22bba40eff9a is approved.
you: █
```

Unknown ID

```
you: check 00000000-0000-0000-0000-000000000000
bot: Reservation not found.
you: █
```

Stage 1. RAG chatbot

Guardrails and PII protection:

How it works?

Static documents

- Presidio detection
- PII redaction
- Safe ingestion into Weaviate

User input

- LangChain PII Middleware
- LLM processing
- Output middleware
- Additional Presidio response filter

Tests:

- detect sensitive data in text containing an email and phone number
- does not flag a clean parking-information question
- replaces a detected person name with <PERSON>
- redacts PII from every document before Weaviate ingestion
- returns Presidio matches in LangChain-compatible format

rag_chain.py

```
self.middleware = [  
    PIIMiddleware(  
        "email",  
        strategy="redact",  
        apply_to_input=True,  
        apply_to_output=True),  
    PIIMiddleware(  
        "credit_card",  
        strategy="redact",  
        apply_to_input=True,  
        apply_to_output=True),  
    PIIMiddleware(  
        "ip",  
        strategy="redact",  
        apply_to_input=True,  
        apply_to_output=True),  
    PIIMiddleware(  
        "presidio_pii",  
        detector=self.guardrails.detect_for_langchain,  
        strategy="redact",  
        apply_to_input=True,  
        apply_to_output=True)]
```

vector_store.py

```
if guardrails is not None:  
    texts = guardrails.filter_documents_for_ingestion(texts)
```

```
plugins: anyio-4.14.1, langsmith-0.9.5, locust-2.44.4, mock-3.14.0  
collected 5 items  
  
tests/test_guardrails.py::test_detects_email_and_phone_number PASSED [ 20%]  
tests/test_guardrails.py::test_clean_text_has_no_sensitive_data PASSED [ 40%]  
tests/test_guardrails.py::test_redact_replaces_pii_with_placeholder PASSED [ 60%]  
tests/test_guardrails.py::test_filter_documents_for_ingestion_redacts_each_text PASSED [ 80%]  
tests/test_guardrails.py::test_presidio_detector_returns_langchain_match_format PASSED [100%]  
  
===== 5 passed in 5.12s =====
```

Stage 1. RAG chatbot

Evaluation:

Results saved to both JSON and Markdown

Why Avg Precision@3 is 0.333?

Each query had one labelled relevant document and the retriever returned 3 documents. So one correct document in the top 3 gives $\text{Precision@3} = 1/3$

**a piece of report*

```
## Environment
| Item | Value |
| --- | --- |
| Python | 3.11.5 |
| LangChain | 1.3.11 |
| LLM model | gpt-4o-mini |
| Embedding model | text-embedding-3-small |
| Weaviate collection | ParkingInfo |

## Retrieval quality
Evaluated 7 labelled queries at K=3.

| Metric | Result |
| --- | --- |
| Average Recall@3 | 1.000 |
| Average Precision@3 | 0.333 |

### Per-query results
| Query | Expected | Retrieved | Recall | Precision |
| --- | --- | --- | --- | --- |
| Where are you located? | location-1 | location-1, details-2, general-1 | 1.00 | 0.33 |
| What documents are required to make a reservation? | booking-1 | booking-1, booking-2, policy-1 | 1.00 | 0.33 |
| Can I cancel my reservation? | booking-2 | booking-2, booking-1, policy-1 | 1.00 | 0.33 |
| Do you have electric vehicle charging? | details-2 | details-2, details-1, general-1 | 1.00 | 0.33 |
| What parking zones do you have? | details-1 | details-1, details-2, general-1 | 1.00 | 0.33 |
| Do you share my personal data with anyone? | policy-1 | policy-1, booking-1, general-1 | 1.00 | 0.33 |
| Tell me about your facility security. | general-1 | general-1, details-2, details-1 | 1.00 | 0.33 |

## Performance
| Metric | Result |
| --- | --- |
| Requests attempted | 15 |
| Requests successful | 15 |
| Success rate | 100.0% |
| Throughput | 0.273 requests/s |
| Total test duration | 54.894 s |

### Retrieval latency
| Metric | Result |
| --- | --- |
| Mean | 0.282 s |
| P50 | 0.196 s |
| P90 | 0.389 s |
| P95 | 0.681 s |
| Maximum | 1.275 s |

### Full answer latency
| Metric | Result |
| --- | --- |
| Mean | 3.377 s |
| P50 | 4.890 s |
| P90 | 5.185 s |
| P95 | 5.791 s |
| Maximum | 6.810 s |
```

Stage 1. RAG chatbot

Automated test coverage:

test_chatbot.py
test_database.py
test_eval_latency.py
test_eval_retrieval.py
test_guardrails.py
test_rag_chain.py
test_reservation_validator.py
test_vector_store.py

What is being tested:

- reservation-field extraction
- validation rules
- RAG formatting and retrieval
- guardrail mechanism
- Weaviate ingestion
- evaluation calculations

```
collected 43 items
tests/test_chatbot.py::test_extract_slots_parses_car_number_and_dates PASSED [ 2%]
tests/test_chatbot.py::test_ask_for_missing_lists_unfilled_slots PASSED [ 4%]
tests/test_chatbot.py::test_collect_reservation_creates_db_row_once_complete PASSED [ 6%]
tests/test_chatbot.py::test_collect_reservation_asks_for_missing_info_when_incomplete PASSED [ 9%]
tests/test_chatbot.py::test_collect_reservation_rejects_short_names PASSED [ 11%]
tests/test_chatbot.py::test_collect_reservation_reports_no_available_space PASSED [ 13%]
tests/test_chatbot.py::test_handle_message_invokes_compiled_graph PASSED [ 16%]
tests/test_chatbot.py::test_graph_preserves_session_id_across_messages PASSED [ 18%]
tests/test_chatbot.py::test_graph_routes_information_request_to_rag PASSED [ 20%]
tests/test_chatbot.py::test_graph_reset_command_clears_active_session PASSED [ 23%]
tests/test_chatbot.py::test_new_reservation_request_resets_existing_session_through_graph PASSED [ 25%]
tests/test_chatbot.py::test_extract_slots_fills_start_and_end_across_separate_messages PASSED [ 27%]
tests/test_chatbot.py::test_graph_collects_dates_across_separate_messages PASSED [ 30%]
tests/test_database.py::test_get_available_spaces_filters_unavailable PASSED [ 32%]
tests/test_database.py::test_find_available_space_ignores_conflicting_space PASSED [ 34%]
tests/test_database.py::test_find_available_space_returns_none_when_all_conflict PASSED [ 37%]
tests/test_database.py::test_create_and_get_reservation_round_trip PASSED [ 39%]
tests/test_database.py::test_update_reservation_status PASSED [ 41%]
tests/test_eval_latency.py::test_evaluate_performance_success PASSED [ 44%]
tests/test_eval_latency.py::test_evaluate_performance_failure PASSED [ 46%]
tests/test_eval_retrieval.py::test_evaluate_retrieval_returns_summary PASSED [ 48%]
tests/test_eval_retrieval.py::test_evaluate_retrieval_uses_requested_k PASSED [ 51%]
tests/test_guardrails.py::test_detects_email_and_phone_number PASSED [ 53%]
tests/test_guardrails.py::test_clean_text_has_no_sensitive_data PASSED [ 55%]
tests/test_guardrails.py::test_redact_replaces_pii_with_placeholder PASSED [ 58%]
tests/test_guardrails.py::test_filter_documents_for_ingestion_redacts_each_text PASSED [ 60%]
tests/test_guardrails.py::test_presidio_detector_returns_langchain_match_format PASSED [ 62%]
tests/test_rag_chain.py::test_format_docs_with_documents PASSED [ 65%]
tests/test_rag_chain.py::test_format_docs_empty_list PASSED [ 67%]
tests/test_rag_chain.py::test_format_dynamic_includes_spaces_and_hours PASSED [ 69%]
tests/test_rag_chain.py::test_rag_chain_creates_real_langchain_agent_with_pii_middleware PASSED [ 72%]
tests/test_rag_chain.py::test_rag_chain_invokes_guarded_agent_and_filters_final_response PASSED [ 74%]
tests/test_reservation_validator.py::test_rejects_short_name_and_surname PASSED [ 76%]
tests/test_reservation_validator.py::test_rejects_invalid_car_number PASSED [ 79%]
tests/test_reservation_validator.py::test_rejects_past_reservation PASSED [ 81%]
tests/test_reservation_validator.py::test_rejects_end_before_start PASSED [ 83%]
tests/test_reservation_validator.py::test_rejects_too_long_reservation PASSED [ 86%]
tests/test_reservation_validator.py::test_rejects_period_outside_working_hours PASSED [ 88%]
tests/test_reservation_validator.py::test_accepts_valid_reservation PASSED [ 90%]
tests/test_vector_store.py::test_ensure_schema_skips_creation_when_class_exists PASSED [ 93%]
tests/test_vector_store.py::test_ensure_schema_creates_class_when_missing PASSED [ 95%]
tests/test_vector_store.py::test_ingest_from_json_applies_guardrails PASSED [ 97%]
tests/test_vector_store.py::test_retriever_delegates_to_store PASSED [100%]
===== 43 passed in 5.97s =====
```

Stage 2. Human-in-the-loop admin agent

After Stage 1 creates a pending reservation, the chatbot sends its reservation ID to the authenticated administrator REST API. A separate LangChain agent receives the administrator instruction and can propose one of the actions:

- approve_reservation
- refuse_reservation

HumanInTheLoopMiddleware interrupts execution before either tool runs. The proposed action is executed only after explicit human confirmation

admin_agent.py

```
middleware=[
    force_admin_action_tool,
    HumanInTheLoopMiddleware(
        interrupt_on={
            "approve_reservation": {
                "allowed_decisions": [
                    "approve",
                    "reject"],
                "description": (
                    "Confirm execution of reservation approval")),
            "refuse_reservation": {
                "allowed_decisions": [
                    "approve",
                    "reject"],
                "description": (
                    "Confirm execution of reservation refusal"))},
    ]
```

```
self.tools = [
    tool
    for tool in all_tools
    if tool.name
    in {
        "approve_reservation",
        "refuse_reservation"}]
```

Stage 2. Reservation escalation

When the chatbot creates a valid reservation:

- the reservation is saved in PostgreSQL with status pending
- **AdminClient** sends POST /admin/requests
- the API creates an **AdminReview** with state pending_admin
- the chatbot confirms that the request is pending administrator approval

```
City/Zone Assistant ready. Type 'Exit' to quit.
you: I want to make a reservation
bot: To complete your reservation I still need: your first name, your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YY-MM-DD HH:MM).
you: Anna
bot: To complete your reservation I still need: your surname, your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: Ponomarenko
bot: To complete your reservation I still need: your car registration number, the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: AA5678AA
bot: To complete your reservation I still need: the reservation start date/time (YYYY-MM-DD HH:MM), the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-08-25 10:30
bot: To complete your reservation I still need: the reservation end date/time (YYYY-MM-DD HH:MM).
you: 2026-08-25 13:30
bot: Thanks Anna, your reservation request for Zone B was created. The reservation is pending administrator approval. Reservation reference: 806f59b0-3ba1-4870-82c1-a6771655beb7
you: █
```

Starting the administrator API:

```
(venv) @MacBook-Air-Viktoria AI_Engineering_Practical_Task konia % python -m uvicorn src.admin_api:app --host 127.0.0.1 --port 8000
INFO: Started server process [37993]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

list pending:

```
{
  "reservation_id": "806f59b0-3ba1-4870-82c1-a6771655beb7",
  "thread_id": "82bc20a7-43c3-4568-92d5-f8df5be9c62b",
  "review_state": "pending_admin",
  "proposed_action": null,
  "admin_comment": null,
  "created_at": "2026-08-28T11:56:21.792425+00:00",
  "decided_at": null,
  "reservation": {
    "name": "Anna",
    "surname": "Ponomarenko",
    "car_number": "AA5678AA",
    "period_start": "2026-08-25T10:30:00",
    "period_end": "2026-08-25T13:30:00",
    "status": "pending",
    "parking_space_id": 2
  }
}
```

The same reservation ID appears in both the chatbot response and the administrator queue, proving communication between the two agents.

Stage 2. Administrator review state

Administrator reviews are stored in PostgreSQL. Each review contains:

- reservation ID
- LangGraph thread ID
- review state
- proposed action
- administrator comment
- created and decided timestamps
- reservation details

```
admin> open 806f59b0-3ba1-4870-82c1-a6771655beb7
{
  "reservation_id": "806f59b0-3ba1-4870-82c1-a6771655beb7",
  "thread_id": "82bc20a7-43c3-4568-92d5-f8df5be9c62b",
  "review_state": "pending_admin",
  "proposed_action": null,
  "admin_comment": null,
  "created_at": "2026-06-28T11:56:21.792425+00:00",
  "decided_at": null,
  "reservation": {
    "name": "Anna",
    "surname": "Ponomarenko",
    "car_number": "AA5678AA",
    "period_start": "2026-08-25T10:30:00",
    "period_end": "2026-08-25T13:30:00",
    "status": "pending",
    "parking_space_id": 2
  }
}
Decision instruction (for example: 'approve this reservation'): █
```

Review-state transitions:

pending_admin → awaiting_confirmation → approved or refused

Stage 2. Approval workflow

The Stage 2 admin agent proposes `approve_reservation` or `refuse_reservation`, but the action is not executed immediately

HumanInTheLoopMiddleware returns an `awaiting_confirmation` state with:

- proposed tool name
- tool arguments
- allowed human decisions (approve / reject)

This proves that the administrator remains in control before any state-changing tool is executed

```
admin> open 806f59b0-3ba1-4870-82c1-a6771655beb7
{
  "reservation_id": "806f59b0-3ba1-4870-82c1-a6771655beb7",
  "thread_id": "82bc20a7-43c3-4568-92d5-f8df5be9c62b",
  "review_state": "pending_admin",
  "proposed_action": null,
  "admin_comment": null,
  "created_at": "2026-06-28T11:56:21.792425+00:00",
  "decided_at": null,
  "reservation": {
    "name": "Anna",
    "surname": "Ponomarenko",
    "car_number": "AA5678AA",
    "period_start": "2026-08-25T10:30:00",
    "period_end": "2026-08-25T13:30:00",
    "status": "pending",
    "parking_space_id": 2
  }
}
Decision instruction (for example: 'approve this reservation'): approve this reservation
{
  "status": "awaiting_confirmation",
  "thread_id": "82bc20a7-43c3-4568-92d5-f8df5be9c62b",
  "reservation_id": "806f59b0-3ba1-4870-82c1-a6771655beb7",
  "action_requests": [
    {
      "name": "approve_reservation",
      "args": {
        "reservation_id": "806f59b0-3ba1-4870-82c1-a6771655beb7"
      },
      "description": "Confirm execution of reservation approval"
    }
  ],
  "review_configs": [
    {
      "action_name": "approve_reservation",
      "allowed_decisions": [
        "approve",
        "reject"
      ]
    }
  ]
}
```

- the selected tool is `approve_reservation`
- `reservation_id` is passed as tool argument
- execution is interrupted before the tool runs
- only `approve/reject` are accepted as human decisions

Stage 2. REST API and persistence

The chatbot and administrator agent communicate through an authenticated FastAPI service. The administrator API provides:

- **POST /admin/requests** (escalate a reservation)
- **GET /admin/requests** (list admin reviews)
- **GET /admin/requests/{reservation_id}** (inspect one review)
- **POST /admin/commands** (ask the agent to propose an action)
- **POST /admin/decisions** (submit the human decision)

Persistence:

- Admin reviews are stored in PostgreSQL
- PostgresSaver stores the LangGraph thread state
- the same thread_id is reused when the interrupted execution is resumed

CityPark Administrator API 2.0.0 OAS 3.1
/openapi.json

default

GET	/health	Health	⌵
POST	/admin/requests	Submit Request	⌵
GET	/admin/requests	List Requests	⌵
GET	/admin/requests/{reservation_id}	Get Request	⌵
POST	/admin/commands	Propose Action	⌵
POST	/admin/decisions	Review Action	⌵

Stage 2. Alternative decision paths

Both business actions are protected by the same HITL checkpoint:

- approve_reservation → awaiting_confirmation
- refuse_reservation → awaiting_confirmation

Important:

approve/reject are middleware decisions:

- approve = execute the proposed tool
- reject = cancel the proposed tool execution

```
admin> open 2828ab04-fed8-4a0c-ba2f-8e07090159c2
{
  "reservation_id": "2828ab04-fed8-4a0c-ba2f-8e07090159c2",
  "thread_id": "e436abed-62cf-47d9-bd63-4e7c4417d69c",
  "review_state": "pending_admin",
  "proposed_action": null,
  "admin_comment": null,
  "created_at": "2026-06-28T12:03:17.877133+00:00",
  "decided_at": null,
  "reservation": {
    "name": "Anna",
    "surname": "Ponomarenko",
    "car_number": "AA5678AA",
    "period_start": "2026-08-26T10:30:00",
    "period_end": "2026-08-26T13:30:00",
    "status": "pending",
    "parking_space_id": 1
  }
}
Decision instruction (for example: 'approve this reservation'): refuse this reservation
{
  "status": "awaiting_confirmation",
  "thread_id": "e436abed-62cf-47d9-bd63-4e7c4417d69c",
  "reservation_id": "2828ab04-fed8-4a0c-ba2f-8e07090159c2",
  "action_requests": [
    {
      "name": "refuse_reservation",
      "args": {
        "reservation_id": "2828ab04-fed8-4a0c-ba2f-8e07090159c2"
      },
      "description": "Confirm execution of reservation refusal"
    }
  ],
  "review_configs": [
    {
      "action_name": "refuse_reservation",
      "allowed_decisions": [
        "approve",
        "reject"
      ]
    }
  ]
}
```

Stage 2. Automated test coverage

Automated tests cover:

- admin agent configuration
- approve_reservation and refuse_reservation tools
- authenticated admin API
- reservation escalation
- admin CLI command parsing
- PostgreSQL review-state transitions
- HITL interrupt structure
- approve/refuse workflow logic
- evaluation summary generation

```
(venv) @MacBook-Air-Viktorii AI_Engineering_Practical_Task konia % python
-m pytest \
  tests/test_admin_agent.py \
  tests/test_admin_api.py \
  tests/test_admin_cli.py \
  tests/test_admin_client.py \
  tests/test_admin_tools.py \
  tests/test_stage2_database.py \
  tests/test_stage2_integration.py \
  tests/test_eval_admin_workflow.py \
  -v

===== test session starts =====
platform darwin -- Python 3.11.5, pytest-8.3.3, pluggy-1.6.0 -- /Users/
sktop/AI_Engineering_Practical_Task konia/venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/
configfile: pytest.ini
plugins: anyio-4.14.1, langsmith-0.9.5, locust-2.44.4, mock-3.14.0
collected 21 items

tests/test_admin_agent.py::test_agent_uses_real_human_in_the_loop_middleware PASSED [ 4%]
tests/test_admin_agent.py::test_propose_returns_real_interrupt_details PASSED [ 9%]
tests/test_admin_api.py::test_admin_api_rejects_missing_token PASSED [ 14%]
tests/test_admin_api.py::test_submit_request_creates_admin_review PASSED [ 19%]
tests/test_admin_cli.py::test_parse_confirmation_accepts_yes_and_y PASSED [ 23%]
tests/test_admin_cli.py::test_parse_confirmation_accepts_no_and_n PASSED [ 28%]
tests/test_admin_cli.py::test_parse_confirmation_rejects_unknown_value PASSED [ 33%]
tests/test_admin_cli.py::test_parse_list_command_supports_required_modes PASSED [ 38%]
tests/test_admin_cli.py::test_pending_filter_includes_both_unfinished_states PASSED [ 42%]
tests/test_admin_cli.py::test_all_filter_returns_every_review PASSED [ 47%]
tests/test_admin_client.py::test_submit_reservation_posts_to_admin_api PASSED [ 52%]
tests/test_admin_client.py::test_submit_reservation_wraps_http_errors PASSED [ 57%]
tests/test_admin_tools.py::test_approve_tool_updates_database PASSED [ 61%]
tests/test_admin_tools.py::test_refuse_tool_updates_database PASSED [ 66%]
tests/test_stage2_database.py::test_create_admin_review_is_idempotent PASSED [ 71%]
tests/test_stage2_database.py::test_resolve_admin_review_updates_reservation PASSED [ 76%]
tests/test_stage2_integration.py::test_stage2_approve_flow_end_to_end PASSED [ 80%]
tests/test_stage2_integration.py::test_stage2_refuse_flow_end_to_end PASSED [ 85%]
tests/test_eval_admin_workflow.py::test_summary_calculates_statistics PASSED [ 90%]
tests/test_eval_admin_workflow.py::test_workflow_evaluation_reports_success PASSED [ 95%]
tests/test_eval_admin_workflow.py::test_workflow_evaluation_records_failure PASSED [100%]

===== 21 passed in 1.26s =====
```

Stage 2. Automated evaluation

The evaluation runs the Stage 2 administrator workflow three times and verifies:

- reservation escalation to the admin API
- admin-agent proposal of a protected action
- HITL resume after human confirmation
- successful workflow completion

***Note:** MCP recording is disabled for this Stage 2 evaluation purposes as MCP is introduced in Stage 3

(this can be done by removing line `mcp_recorder=MCPReservationRecorder(resolved_settings)` from `src/admin_api.py`)

```
(venv) @MacBook-Air-Viktoria AI_Engineering_Practical_Task konia % python
-m evaluation.eval_admin_workflow
Runs attempted: 3
Runs successful: 3
Success rate: 100.0%

Request escalation latency:
Mean: 1.472s
P50: 1.439s
Max: 1.601s

Admin-agent proposal latency:
Mean: 3.882s
P50: 3.745s
Max: 4.580s

HITL resume latency:
Mean: 2.950s
P50: 3.236s
Max: 3.273s

JSON results: /Users/ /Desktop/AI_Engineering_Practical_Task konia/evaluati
on/results/admin_workflow_results.json
```

```
app.state.admin_agent = (
    admin_agent
    or AdminApprovalAgent(
        settings=resolved_settings,
        database=resolved_database,
        mcp_recorder=MCPReservationRecorder(
            resolved_settings)))
```

Stage 3. MCP recording layer

Stage 3 adds an MCP recording layer for approved reservations. After a reservation is approved by the administrator, the admin tool updates PostgreSQL and then calls **MCPReservationRecorder**. The MCP recorder sends only the reservation ID to a separate MCP server. The MCP server is responsible for verifying and recording the approved reservation.

approve_reservation → database.resolve_admin_review(...) → **MCPReservationRecorder** → MCP server → approved_reservations.txt

***Note:** MCP_ACCESS_TOKEN is short-lived and needs to be generated at this stage (this can be done with `python -m src.mcp_token`)

```
17     @tool
18     def approve_reservation(reservation_id, comment=""):
19         """Approve a pending parking reservation after human confirmation."""
20         result = database.resolve_admin_review(
21             reservation_id=reservation_id,
22             decision="approved",
23             comment=comment or None)
24         if mcp_recorder is not None:
25             result["mcp_recording"] = mcp_recorder.record(reservation_id)
26
27         return json.dumps(result, default=str)
```

Stage 3. MCP server security

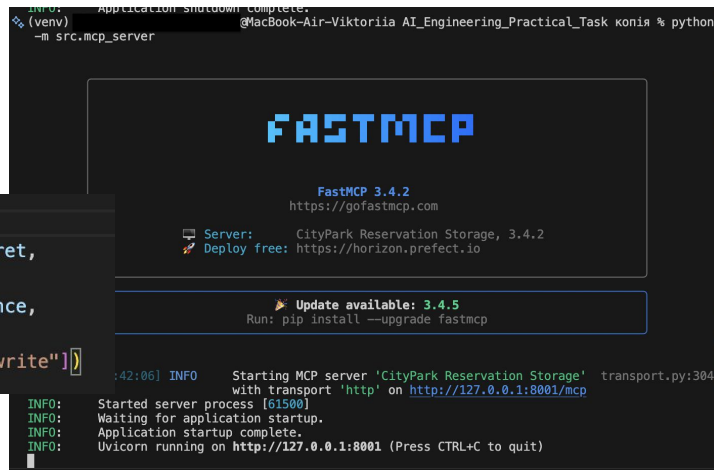
The MCP server is protected by JWT-based authentication. A request is accepted only when the token matches the configured:

- JWT secret
- Issuer
- audience
- required write scope

This prevents unauthorised clients from writing reservations to the output file

mcp_server.py

```
18     verifier = JWTVerifier(  
19         public_key=settings.mcp_jwt_secret,  
20         issuer=settings.mcp_jwt_issuer,  
21         audience=settings.mcp_jwt_audience,  
22         algorithm="HS256",  
23         required_scopes=["reservations:write"]  
24     )
```



```
Application shutdown complete.  
@MacBook-Air-Viktoria AI_Engineering_Practical_Task konia % python  
-m src.mcp_server  
  
FASTMCP  
  
FastMCP 3.4.2  
https://gofastmcp.com  
  
Server: CityPark Reservation Storage, 3.4.2  
Deploy free: https://horizon.prefect.io  
  
Update available: 3.4.5  
Run: pip install --upgrade fastmcp  
  
[42:06] INFO Starting MCP server 'CityPark Reservation Storage' transport.py:304  
INFO: Started server process [61500]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
```

Stage 3. Approved-only recording rule

- the MCP server does not blindly trust the client.
It receives a reservation ID, reads the corresponding review from PostgreSQL, and records the reservation only if the review is approved
- pending or refused reservations are rejected and are not written to the file

mcp_server.py

```
32 @server.tool()
33 def record_approved_reservation(reservation_id):
34     """Record an approved reservation in durable text-file storage."""
35     review = database.get_admin_review(reservation_id)
36     if review is None:
37         raise ValueError("Reservation review was not found")
38     if review["review_state"] != "approved":
39         raise PermissionError(
40             "Only administrator-approved reservations may be recorded")
```

The MCP client sends only reservation_id, the server re-checks the database state itself before writing

Stage 3. File output format

Approved reservations are written to storage/**approved_reservations.txt** and each line follows the required format:

Name Surname | Car Number | Start Time to End Time | Approval Time

```
(venv) @MacBook-Air-Viktoriia AI_Engineering_Practical_Task копія % cat storage/approved_reservations.txt
Anna Ponomarenko | AA5678AA | 2026-08-24T10:00:00 to 2026-08-24T13:30:00 | 2026-06-28T13:43:32.178452+00:00
```

Stage 3. Reliability and idempotency

The file writer is designed to avoid duplicate and corrupted writes. Reliability mechanisms include:

- file locking for concurrent writes
- reservation ID index for duplicate detection
- already_recorded status for repeated calls
- atomic index replacement
- flush and fsync after writing
- retry logic in the MCP client

```
src > mcp_client.py > MCPReservationRecorder > arecord
8 class MCPReservationRecorder:
...
30     async def arecord(self, reservation_id):
31         async for attempt in AsyncRetrying(
32             stop=stop_after_attempt(self.settings.mcp_retry_attempts),
```

idempotency

safe writing

retry

```
src > approved_reservation_store.py > ... 84
10 class ApprovedReservationStore:
54     def record(self, reservation_id, reservation, approval_time):
59
60         with FileLock(str(self.lock_file), timeout=10):
61             recorded_ids = self._load_index()
62             if reservation_id in recorded_ids:
63                 return {
64                     "Status": "already_recorded",
65                     "reservation_id": reservation_id,
66                     "file": str(self.output_file),
67                     "line": line}
68
```

```
81         with self.output_file.open("a", encoding="utf-8") as output:
82             output.write(line + "\n")
83             output.flush()
84             os.fsync(output.fileno())
```

Stage 3. Tests and evaluation

Automated tests verify:

- MCP server authentication
- approved-only recording rule
- file output format
- idempotent recording
- duplicate prevention
- MCP client retry and timeout handling
- admin approval integration with MCP recording

```
(venv) @MacBook-Air-Viktorii AI_Engineering_Practical_Task konia % python -m evaluation.eval
{
  "runs_attempted": 3,
  "runs_successful": 3,
  "success_rate_percent": 100.0,
  "mcp_recording_latency": {
    "mean_seconds": 1.2599185280074987,
    "p50_seconds": 1.503570792003302,
    "max_seconds": 1.654157542012399
  },
  "failures": []
}
JSON results: /Users/ /Desktop/AI_Engineering_Practical_Task konia/evaluation/results/mcp_workflow_results.json
```

```
(venv) @MacBook-Air-Viktorii AI_Engineering_Practical_Task konia % python -m pytest \
tests/test_mcp_server.py \
tests/test_mcp_client.py \
tests/test_approved_reservation_store.py \
tests/test_stage3_admin_tools.py \
tests/test_eval_mcp_workflow.py \
-v
===== test session starts =====
platform darwin -- Python 3.11.5, pytest-8.3.3, pluggy-1.6.0 -- /Users/ /Desktop/AI_Engineering_Practical_Task konia/venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/ /Desktop/AI_Engineering_Practical_Task konia
configfile: pytest.ini
plugins: anyio-4.14.1, langsmith-0.9.5, locust-2.44.4, mock-3.14.0
collected 10 items

tests/test_mcp_server.py::test_server_registers_recording_tool PASSED [ 10%]
tests/test_mcp_server.py::test_recording_logic_rejects_nonapproved_reservation PASSED [ 20%]
tests/test_mcp_client.py::test_client_invokes_named_mcp_tool PASSED [ 30%]
tests/test_mcp_client.py::test_client_fails_when_tool_is_missing PASSED [ 40%]
tests/test_approved_reservation_store.py::test_store_writes_required_format PASSED [ 50%]
tests/test_approved_reservation_store.py::test_store_is_idempotent PASSED [ 60%]
tests/test_stage3_admin_tools.py::test_approve_calls_mcp_after_database_approval PASSED [ 70%]
tests/test_stage3_admin_tools.py::test_refuse_never_calls_mcp PASSED [ 80%]
tests/test_eval_mcp_workflow.py::test_summary_calculates_mcp_latency PASSED [ 90%]
tests/test_eval_mcp_workflow.py::test_summary_handles_no_successful_calls PASSED [100%]

===== 10 passed in 1.05s =====
```

Stage 4. Unified LangGraph orchestration

In this stage, LangGraph orchestration layer that connects all previous components into one workflow is introduced. It reuses:

- Stage 1 RAG chatbot for information queries and reservation collection
- Stage 2 administrator approval agent with HITL confirmation
- Stage 3 MCP recorder for approved reservation storage

The goal is to manage the full reservation lifecycle through a persistent graph-based workflow

Approximate schema:

User request → LangGraph workflow → Stage 1 chatbot → Reservation created? → Stage 2 admin approval → Approved? → Stage 3 MCP recording → Final response

This stage does not replace the previous implementation, it wraps the existing components into a single orchestrated workflow instead

service.py

```
src > orchestration > service.py > build_workflow_service
100 def build_workflow_service(settings=None):
110     guardrails = GuardRails()
111     knowledge_base = StaticKnowledgeBase(vector_client, settings)
112     rag = RAGChain(settings, knowledge_base, database, guardrails)
113     chatbot = ParkingChatbot(
114         settings,
115         rag,
116         database,
117         guardrails,
118         admin_client=None)
119     resources.callback(chatbot.close)
120
121     checkpoint_cm = PostgresSaver.from_conn_string(
122         settings.postgres_checkpoint_dsn)
123     checkpoint = resources.enter_context(checkpoint_cm)
124     checkpoint.setup()
125
126     admin_agent = AdminApprovalAgent(
127         settings=settings,
128         database=database)
129     resources.callback(admin_agent.close)
130     recorder = MCPReservationRecorder(settings)
131
132     graph = build_workflow_graph(
133         OrchestrationNodes(
134             chatbot=chatbot,
135             database=database,
136             admin_agent=admin_agent,
137             mcp_recorder=recorder),
```

Stage 4. Workflow graph and routing

The orchestration is implemented as a LangGraph StateGraph. Main nodes:

- user_interaction
- submit_admin_review
- administrator_approval
- record_reservation
- finalize

The graph routes requests depending on the current state:

- information questions can finish after the chatbot response
- reservation requests continue to administrator review
- approved reservations continue to MCP recording
- refused reservations are finalized without MCP recording

graph.py

```
14 def build_workflow_graph(nodes, checkpoint):
15     """Build and compile the Stage 4 Graph API workflow."""
16     builder = StateGraph(ParkingWorkflowState)
17     builder.add_node("user_interaction", nodes.user_interaction)
18     builder.add_node("submit_admin_review", nodes.submit_admin_review)
19     builder.add_node("administrator_approval", nodes.administrator_approval)
20     builder.add_node(
21         "record_reservation",
22         nodes.record_reservation,
23         retry_policy=RetryPolicy(
24             max_attempts=3,
25             initial_interval=0.5,
26             backoff_factor=2.0,
27             max_interval=4.0,
28             retry_on="retry_external_service")
29     )
30     builder.add_node("finalize", nodes.finalize)
31
32     builder.add_edge(START, "user_interaction")
33     builder.add_conditional_edges(
34         "user_interaction",
35         route_after_user_interaction,
36         {
37             "submit_admin_review": "submit_admin_review",
38             "finalize": "finalize"
39         })
40     builder.add_edge("submit_admin_review", "administrator_approval")
41     builder.add_edge("record_reservation", "finalize")
42     builder.add_edge("finalize", END)
```

Stage 4. Human approval as a graph interrupt

Stage 4 turns administrator approval into a graph-level interrupt

The workflow:

- creates a reservation through the Stage 1 chatbot
- stores the workflow state
- pauses at the administrator approval node
- exposes allowed decisions: approve or refuse
- resumes later using the same workflow_id

```
"state": {
  "workflow_id": "3c9bb0e4-fd92-4055-a0fc-c55be9b3e5a",
  "session_id": "default",
  "user_message": "I want to reserve a parking space. My name is Anna Ponomarenko, car number AA5678AA, from 2026-08-26 10:30 to 2026-08-26 13:30.",
  "assistant_response": "Thanks Anna, your reservation request for Zone B was created. The reservation is pending administrator approval. Reservation reference: e5fcae39-7523-4f79-9664-1536f84fa319",
  "reservation_id": "e5fcae39-7523-4f79-9664-1536f84fa319",
  "reservation": {
    "id": "e5fcae39-7523-4f79-9664-1536f84fa319",
    "parking_space_id": 2,
    "name": "Anna",
    "surname": "Ponomarenko",
    "car_number": "AA5678AA",
    "period_start": "2026-08-26 10:30:00",
    "period_end": "2026-08-26 13:30:00",
    "status": "pending",
    "admin_request_status": "not_sent",
    "admin_request_error": null
  },
  "admin_thread_id": "admin-3c9bb0e4-fd92-4055-a0fc-c55be9b3e5a",
  "recording_status": "not_started",
  "workflow_status": "waiting_for_admin",
  "error": null,
  "interrupt": {
    "interrupt": {
      "type": "reservation_approval", "workflow_id": "3c9bb0e4-fd92-4055-a0fc-c55be9b3e5a", "reservation_id": "e5fcae39-7523-4f79-9664-1536f84fa319", "reservation": {
        "id": "e5fcae39-7523-4f79-9664-1536f84fa319", "parking_space_id": 2, "name": "Anna", "surname": "Ponomarenko", "car_number": "AA5678AA", "period_start": "2026-08-26 10:30:00", "period_end": "2026-08-26 13:30:00", "status": "pending", "admin_request_status": "not_sent", "admin_request_error": None, "question": "Approve or refuse this parking reservation?", "allowed_decisions": ["approve", "refuse"]}, "id": "ad7e304247cf50c3f32ce9c2506ef96"}
    }
  }
}

administrator [approve/refuse]:
```

nodes.py

```
51 def administrator_approval(self, state):
52     """Pause the parent graph, then execute the existing Stage 2 agent."""
53     decision = interrupt(
54         {
55             "type": "reservation_approval",
56             "workflow_id": state["workflow_id"],
57             "reservation_id": state["reservation_id"],
58             "reservation": state["reservation"],
59             "question": "Approve or refuse this parking reservation?",
60             "allowed_decisions": ["approve", "refuse"]}
61     if not isinstance(decision, dict):
62         raise ValueError("Administrator decision must be a JSON object")
63
```

service.py

```
41 def resume_admin(self, workflow_id, decision, comment=None):
42     result = self.graph.invoke(
43         Command(
44             resume={
45                 "decision": decision,
46                 "comment": comment}),
47         config=self.config(workflow_id),
48         durability="sync")
49     return self._response(workflow_id, result)
```

Stage 4. Completed workflow with MCP recording

After the administrator approves the interrupted workflow, LangGraph resumes execution and continues to the MCP recording node. The final state confirms:

- administrator decision: approved
- MCP recording status: recorded
- workflow status: completed
- final response returned to the user

```
administrator [approve/refuse]: approve
comment (optional):
{
  "workflow_id": "cb52a6bb-3e65-42e8-914b-3bb5fa6f4141",
  "status": "completed",
  "response": "Reservation 07e7101d-fe40-41fb-a089-ae5d47f3eddc was approved and stored by the MCP server (recorded).",
  "reservation_id": "07e7101d-fe40-41fb-a089-ae5d47f3eddc",
  "interrupts": [],
  "state": {
    "workflow_id": "cb52a6bb-3e65-42e8-914b-3bb5fa6f4141",
    "session_id": "default",
    "user_message": "I want to reserve a parking space. My name is Anna Ponomarenko, car number AA5678AA, from 2026-08-27 10:30 to 2026-08-27 13:30.",
    "assistant_response": "thanks Anna, your reservation request for Zone B was created. The reservation is pending administrator approval. Reservation reference: 07e7101d-fe40-41fb-a089-ae5d47f3eddc",
    "reservation_id": "07e7101d-fe40-41fb-a089-ae5d47f3eddc",
    "reservation": {
      "id": "07e7101d-fe40-41fb-a089-ae5d47f3eddc",
      "parking_space_id": 2,
      "name": "Anna",
      "surname": "Ponomarenko",
      "car_number": "AA5678AA",
      "period_start": "2026-08-27 10:30:00",
      "period_end": "2026-08-27 13:30:00",
      "status": "pending",
      "admin_request_status": "not_sent",
      "admin_request_error": null
    },
    "admin_thread_id": "admin-cb52a6bb-3e65-42e8-914b-3bb5fa6f4141",
    "admin_decision": "approved",
    "admin_comment": null,
    "recording_status": "recorded",
    "recording_result": {
      "status": "recorded",
      "reservation_id": "07e7101d-fe40-41fb-a089-ae5d47f3eddc",
      "file": "/Users/ /Desktop/AI_Engineering_Practical_Task /u043a/u043e/u043f/u0456/u044f/storage/approved_reservations.txt",
      "line": "Anna Ponomarenko | AA5678AA | 2026-08-27T10:30:00 to 2026-08-27T13:30:00 | 2026-08-05T13:39:09.213577+00:00"
    },
    "workflow_status": "completed",
    "final_response": "Reservation 07e7101d-fe40-41fb-a089-ae5d47f3eddc was approved and stored by the MCP server (recorded).",
    "error": null
  }
}
you: 
```

Stage 4. Checkpointing, state and recovery

service.py

```
119 resources.callback(chatbot.close)
120
121 checkpoint_cm = PostgresSaver.from_conn_string(
122     settings.postgres_checkpoint_dsn)
123 checkpointer = resources.enter_context(checkpointer_cm)
124 checkpointer.setup()
125
126 admin_agent = AdminApprovalAgent(
127     settings=settings,
128     database=database)
129 resources.callback(admin_agent.close)
130 recorder = MCPReservationRecorder(settings)
131
```

Each workflow has a workflow_id and persistent state. LangGraph checkpoints allow the workflow to:

- pause while waiting for administrator input
- resume after a human decision
- keep state across API calls
- retry failed post-approval recording without repeating the whole conversation

```
def retry(self, workflow_id):
    result = self.graph.invoke(
        None,
        config=self.config(workflow_id),
        durability="sync")
    return self._response(workflow_id, result)
```

```
def resume_admin(self, workflow_id, decision, comment=None):
    result = self.graph.invoke(
        Command(
            resume={
                "decision": decision,
                "comment": comment}),
        config=self.config(workflow_id),
        durability="sync")
    return self._response(workflow_id, result)
```

This is useful because MCP recording can fail independently from approval, and the workflow can retry the failed step without recreating the reservation

Stage 4. API, CLI and observability

Stage 4 exposes the orchestrated workflow through a FastAPI service and a CLI. The API supports:

- starting a workflow
- submitting administrator decision
- retrying failed workflow steps
- reading current workflow state
- reading workflow history

CityPark Unified LangGraph API

4.0.0

OAS 3.1

/openapi.json

default

GET	/health	Health
POST	/workflows	Start Workflow
POST	/workflows/{workflow_id}/admin-decision	Resume Workflow
POST	/workflows/{workflow_id}/retry	Retry Workflow
GET	/workflows/{workflow_id}	Get Workflow
GET	/workflows/{workflow_id}/history	Get History

Stage 4. Tests and evaluation

Stage 4 was verified with automated tests, workflow evaluation and HTTP load testing.

Automated tests cover:

- graph construction and routing
- orchestration nodes
- workflow service
- workflow API
- admin decision handling
- MCP recording path
- error and retry behavior

python -m evaluation.eval_orchestrated_workflow

```
{
  "runs_attempted": 3,
  "runs_successful": 3,
  "success_rate_percent": 100.0,
  "information_workflow_latency_seconds": {
    "mean": 10.9785,
    "p50": 9.6575,
    "max": 14.186
  },
  "failures": []
}
```

```
(venv) @MacBook-Air-Viktoria AI_Engineering_Practical_Task konia % python -m pytest \
tests/test_orchestration_graph.py \
tests/test_orchestration_nodes.py \
tests/test_orchestration_routing.py \
tests/test_orchestration_service.py \
tests/test_workflow_api.py \
-v
===== test session starts =====
platform darwin -- Python 3.11.5, pytest-8.3.3, pluggy-1.6.0 -- /Users/ /Desktop/AI_Engineering_Practical_Task konia/venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/ /Desktop/AI_Engineering_Practical_Task konia
configfile: pytest.ini
plugins: anyio-4.14.1, langsmith-0.9.5, locust-2.44.4, mock-3.14.0
collected 12 items

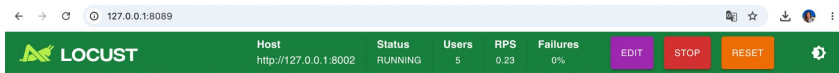
tests/test_orchestration_graph.py::test_information_path_finishes_without_admin_or_mcp PASSED [ 8%]
tests/test_orchestration_graph.py::test_approved_path_interrupts_then_records_through_mcp PASSED [ 16%]
tests/test_orchestration_nodes.py::test_user_interaction_reuses_chatbot_and_loads_reservation PASSED [ 25%]
tests/test_orchestration_nodes.py::test_record_reservation_normalizes_mcp_text_content PASSED [ 33%]
tests/test_orchestration_nodes.py::test_finalize_refused_reservation_does_not_claim_recording PASSED [ 41%]
tests/test_orchestration_nodes.py::test_normalize_mcp_result_accepts_plain_dict PASSED [ 50%]
tests/test_orchestration_routing.py::test_routes_new_reservation_to_admin_review PASSED [ 58%]
tests/test_orchestration_routing.py::test_routes_information_response_to_finalize PASSED [ 66%]
tests/test_orchestration_service.py::test_start_workflow_id_as_langgraph_thread_id PASSED [ 75%]
tests/test_orchestration_service.py::test_resume_admin_uses_command_resume_payload PASSED [ 83%]
tests/test_workflow_api.py::test_start_workflow_endpoint_calls_unified_service PASSED [ 91%]
tests/test_workflow_api.py::test_admin_decision_requires_bearer_token PASSED [100%]

===== 12 passed in 4.66s =====
```

Stage 4. Load testing

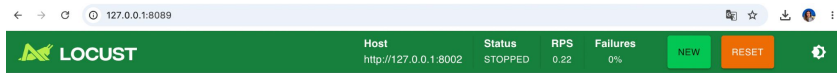
Locust load test against Stage 4 Workflow API:

- host: <http://127.0.0.1:8002>
- 5 concurrent users
- workflow information query and state lookup
- 0% failures



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	workflow: information query	32	0	23000	31000	53000	24670.51	15897	52552	954.25	0.3	0
GET	workflow: state lookup	8	0	660	980	980	641.96	143	979	110	0	0
Aggregated		40	0	23000	31000	53000	19864.8	143	52552	785.4	0.3	0

*Load testing covered information workflows and workflow-state lookups. Administrator approval and MCP recording were covered by integration/evaluation tests



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	workflow: information query	74	0	24000	32000	80000	25806.41	15897	79585	970.08	0.1	0
GET	workflow: state lookup	17	0	650	1100	1100	652.68	143	1071	110	0.1	0
Aggregated		91	0	24000	31000	80000	21107.36	143	79585	809.41	0.2	0

Thank you for your attention!